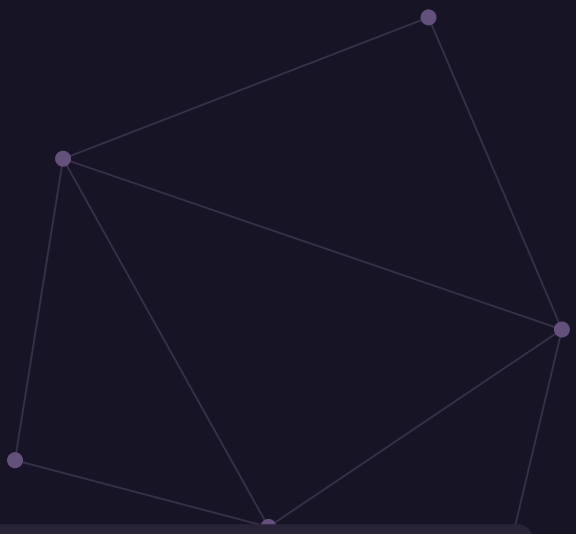
ORBITRON

ORBITRON / SECURITY RESEARCH

php2shell

Tespit ve müdahale rehberi



Laboratuvar bulguları, log imzaları
ve operasyonel müdahale adımları.

CVE-2026-109618

CVE-2026-109619

9.2
CVSS v4.0

v1.2 / TÜRKÇE

21 Ekim 2026

Hazırlayan: ORBITRON Ekibi

00 Genel bakış

CVE-2026-109618 ve CVE-2026-109619 (php2shell; her biri CVSS v4.0 9.2), PHP motorunda bellek tükenmesi ve istekler arasında kalan geçersiz durumla ilişkili iki zafiyettir. **8.2.35 / 8.3.36 / 8.4.27 / 8.5.12** sürümlerinde giderilmiştir. Bu rehber, istek başlangıcındaki tetik koşullarını ve SOC ekiplerinin toplayabileceği izleri ele alır. **Bir log eşleşmesi tek başına saldırı girişimini, yamasız sürümü veya başarılı sömürüyü kanıtlamaz.** İstek bağlamı, süreç telemetrisi ve üreticinin paket bilgisi birlikte değerlendirilmelidir.

01 Kapsam ve doğrulama sınırları

Bu rehber, PHP'nin resmî duyuruları ile ORBITRON'un izole laboratuvar sonuçlarını bir araya getirir. Lab çalışmasında resmî kaynaklardan derlenen yamasız/yamalı sürümler, upstream regresyon testleri ve kontrollü ek testler kullanılmıştır. Üretim sistemlerinde sömürü tetiklenmemiştir.

- **Resmî doğrulama:** Dört yamalı sürüm PHP tarafından yayımlanmıştır. İlgili duyurular GHSA-w8hw-w9fx-j79w / CVE-2026-109618 ve GHSA-6wmf-4rv5-p3rp / CVE-2026-109619'dur. Düzeltme, kaynakta Avoid state corruption via bailout başlığıyla yer alır.
- **Lab doğrulaması:** Aşağıdaki 8.3.35/8.3.36 matrisi, ayrıştırma ve session hata yönetimindeki farkları gösterir. Ayrı 8.4.26/8.4.27 testinde, önceden hazırlanmış preload sınıfı ve session verisi kullanılarak istek başlangıcında kontrollü dosya yazma gösterilmiştir. Bu iki test serisinin sürümleri ve ön koşulları birbirine karıştırılmamalıdır.
- **Kanıt sınırı:** Tam bellek bozulması → RCE zinciri (heap grooming, pointer hijack) geliştirilmemiştir. RCE etkisi PHP güvenlik duyurularında doğrulanır; kontrollü lab dosya yazma testi tek başına hazırlıksız bir sunucuda uzaktan RCE kanıtı değildir.
- **Eski sürümler:** PHP 8.1 ve öncesi normal upstream destek kapsamı dışındadır. Bununla birlikte PHP güvenlik ekibi 7.4, 8.0 ve 8.1 için asgari kaynak kodu backport'ları sağlamıştır. Bunlar normal destek yaşam döngüsünü yeniden başlatmaz. Dağıtım/LTS paketinin iki CVE'yi giderdiğini üreticiden doğrulayın.
- **Kapsam:** PHP'ye göre risk mod_php'de en yüksektir; web isteği karşılayan diğer SAPI'ler de güncellenmelidir. FPM'in başarısız startup sonrasında worker'ı kapatması yalnızca o sürecin sonraki isteğe taşınmasını keser. CVE-2026-109619 için başarılı startup sonrasındaki uygun bir uygulama işlemi ve sonraki istekteki erken tüketici üzerinden FPM yolu da resmî duyuruda açıklanmıştır.
- **Kapsam dışı genellemeler:** Her tarihsel PHP sürümü ve her CMS için aynı sömürü koşulları bu çalışmada bağımsız olarak doğrulanmamıştır.

02 Zafiyet nasıl çalışıyor

PHP, uygun içerik türlerinde istek gövdesini uygulama betiği çalışmadan önce otomatik ayrıştırabilir. Rehberde incelenen yollar:

1. **Bellek tüketimi:** Çok sayıda multipart başlığı, uzun/iç içe alan adları veya kullanılabilir bir `Content-Length` olmadan işlenen veri, küçük görünen bir isteğin ayrıştırma maliyetini büyütebilir. Yamasız ayrıştırıcıda bazı yollarda `post_max_size` denetimi aşılabilir. 2. **CVE-2026-109618:** Bir bellek tahsisi başarısız olduğunda `zend_bailout()` ile erken çıkış, geçici olarak erişilebilir fakat henüz tamamlanmamış iç veriyi bırakabilir. Sonraki temizleme işlemi bu veriyi kullanabilir. Startup dışındaki uygulama yolları da OOM üretebilir. 3.

CVE-2026-109619: Bir isteğe ait bellek serbest bırakıldıktan sonra worker'ın global durumu bu belleğe işaret etmeye devam edebilir. Aynı worker'daki sonraki isteğin erken işlemleri, normal başlatma bunu sıfırlamadan önce kullanabilir. OOM dışında fatal hatalar ve bazı kapanış tanıları da ilgili durumun kalmasına yol açabilir; her iki CVE zorunlu olarak tek bir zincirin ardışık adımları değildir.

Yama; güvenli noktalara ertelenen OOM işleme (`EG(delayed_oom)`), başarısızlık döndürebilen tahsis yolları, ayrıştırıcı kontrolleri ve ilgili global durumun sıfırlanmasını içerir. Upload-progress deserialize yolu da sınıf çözümlemeyen **incomplete-classes-only** moduna alınır. Bu, PHP'nin artık hiç OOM üretmeyeceği veya tüm `longjmp` kullanımının kaldırıldığı anlamına gelmez.

Aşağıdaki tablo yalnızca lab'de incelenen startup-OOM senaryosunu özetler; genel bağışıklık tablosu değildir.

SAPI	Startup-OOM sonucu	Gözlenen kayıt	Aynı worker'ın yeniden kullanımı
PHP-FPM	Worker sonlanır (`FPM_EXIT_SOFTWARE`)	`child <PID> exited with code 70` ve OOM hatası	Sonlanan worker yeniden kullanılmaz; diğer yollar dışlanmaz
mod_php	Lab'de istek 500 döndü, worker devam etti	Apache error log'da OOM	Aynı PID yeniden kullanıldı; bu gözlem tek başına UAF kanıtı değildir
CGI: istek başına süreç	Süreç sonlanır	Startup-OOM hatası	Sonlanan süreç için sonraki istek yoktur; aynı istek içindeki risk dışlanmaz

`code 70` bir SIGSEGV değildir. Başka startup hataları da bu çıkış kodunu üretebilir; OOM nedenini PHP hata kaydıyla doğrulayın.

03 Lab doğrulaması

Yamasız ve yamalı sürümler debug + AddressSanitizer ile derlendi; PHP'nin yeni regresyon testleri her ikisinde çalıştırıldı.

Kanıt matrisi (PHP'nin kendi testleri; 8.3.35 yamasız, 8.3.36 yamalı):

Test	Yamasız	Yamalı	Ne gösterir
Resmi regresyon testleri (toplam)	27'den 5'i başarısız	27/27 geçer	Seçili regresyon kümesi
Multipart `post_max_size` bypass (GHSA-2cv3)	FAIL - bütün gövde `$_POST`'a alınır	PASS - uyarı + veri atılır	Kontrollü OOM giriş primitifi
600.000 yinelenen part başlığı (GHSA-f883 007)	FAIL - sınırsız birikim → OOM	PASS - yalnızca 2 başlık saklanır	Bellek tüketim vektörü
Upload-progress, var olmayan betik (`rfc1867_no_user_code`)	FAIL - sınıf çözümlenir, `$_SESSION` geride kalır	PASS - testte beklenen userland kodu çalışmaz	RINIT-öncesi kod çalıştırma
Geçersiz SID + upload-progress (`rfc1867_sid_invalid`)	FAIL - okuma başarısızken yazma denenir	PASS - hiçbir şey yazmaz	A4 imzasının kaynağı (aşağıda)

Önlem matrisi - hangi geçici ayar OOM'u gerçekten durduruyor (yamasız 8.3.35 php-cgi, PHP'nin kendi test gövdesiyle):

Ayar	Sonuç
Varsayılan (memory_limit=128M)	OOM - `in Unknown on line 0`
`post_max_size=8M` tek başına	OOM (durdurmaz)
`session.upload_progress.enabled=0` tek başına	OOM (durdurmaz)
`session.use_strict_mode=1` tek başına	OOM (durdurmaz)
`memory_limit=-1`	Tamamlanır (~218M tepe)
Yamalı 8.3.36, varsayılan	Tamamlanır (~2M tepe)

Bu test gövdesi ve yapılandırma için, denenen ayarlar arasında `memory_limit=-1` PHP bellek limiti kaynaklı kesilmeyi önlemiştir. Bu genel bir koruma garantisi değildir: altta yatan tahsis başarısızlığı hâlâ mümkündür. PHP limitinin kaldırılması DoS riskini artırır; cgroup/container sınırları, istek ve eşzamanlılık limitleriyle kaynak tüketimini sınırlayın. Cgroup sınırında worker/container OOM-kill olabilir; bu mekanizma PHP'nin `memory_limit` hatasıyla aynı değildir. `RLIMIT_AS`/`RLIMIT_DATA` türü limitleri bunun eşdeğeri saymayın. Rakamlar debug + ASan derlemelerindeki ölçümlerdir; üretim bellek ihtiyacını doğrudan temsil etmez. ORBITRON ayrıca dağıtımın 8.3.6 mod_php derlemesinde aynı gövdenin 128M sınırını tükettiğini gözlemlemiştir.

04 Loglar nerede bulunur

Aşağıdaki yollar yaygın örneklerdir; dağıtım, kurulum, container ve vhost yapılandırmasına göre değişir. **Gerçek yolu çalışan servisin yapılandırmasından doğrulayın.**

Kurulum / kaynak	Toplanacak kayıtlar	Ne için
Rocky / RHEL / Alma - Apache	`/var/log/httpd/error_log`, `/var/log/httpd/access_log`; vhost logları	PHP/Apache hataları + ilişkili istekler
Ubuntu / Debian - Apache	`/var/log/apache2/error.log`, `access.log`; vhost logları	Aynı; `ErrorLog`/`CustomLog` belirleyici
nginx	`/var/log/nginx/error.log`, `access.log`	FastCGI/upstream hataları, 5xx, istek bağlamı
Rocky/RHEL - PHP-FPM	`/var/log/php-fpm/error.log` + pool PHP hata dosyası (örn. `www-error.log`)	FPM süreç olayları + PHP fatal/warning
Ubuntu/Debian - PHP-FPM	`/var/log/php8.4-fpm.log` (sürüme göre ad değişir) + pool hata dosyası	Worker sonlanmaları + PHP hataları
systemd	`journalctl -u php\*-fpm\* -u httpd -u apache2 -u nginx`	Servis hataları, restart'lar, journal çıktısı
Container / Kubernetes	Container stdout/stderr + içerideki ayrı log dosyaları	Pod/container/node bilgisiyle PHP/FPM/web logları
WAF / reverse proxy	Güvenlik olayları, kural eşleşmeleri, body-inspection hataları	Multipart anomalileri, engelleme
Endpoint	EDR olayları; auditd varsa `/var/log/audit/audit.log`	PHP/web worker kaynaklı süreç/dosya/ağ etkinliği

Önemli ayrım: FPM'in ana `error_log` ayarı ile PHP'nin `error_log` ayarı **farklı** kayıtları yönetir. Yalnızca birini toplamak diğerindeki olayları kaçırır. nginx access log'u toplamak, PHP hata log'unu toplamak anlamına gelmez.

Gerçek yolu belirleme:

- **Apache:** etkin sunucu ve vhost'taki `ErrorLog` / `CustomLog`.
- **nginx:** etkin `http`/`server`/`location` bloklarındaki `error_log` / `access_log`.
- **FPM:** `php-fpm.conf`'taki `error_log`; pool'daki `php_admin_value[error_log]`, `php_admin_flag[log_errors]`, `access.log`.
- **PHP:** web isteğini karşılayan SAPI'nin yüklediği php.ini. CLI'daki `php --ini` çıktısının web SAPI ile aynı olduğunu **varsaymayın**.

Ön koşul: `log_errors=On` ve `error_log` tanımlı olmalı. FPM'de worker stdout/stderr'in ana log'a taşınması için `catch_workers_output=yes` önerilir. `display_errors=Off` olsa bile fatal'ler log'a düşer.

05 Tetik imzaları ve log örnekleri

Aşağıdaki ham log satırları yamasız 8.3.x FPM ve mod_php labında gerçek vektör tetiklenerek üretilmiştir. Bunlar bağlamla değerlendirilmesi gereken **avcılık sinyalleridir**; kısaltılmış satırlarda `...` kullanılmıştır.

A1 - Ana tetik: istek ayrıştırılırken OOM (CVE-2026-109618)

FPM (stderr ana log'a taşınmış):

LOG / PATTERN

```
WARNING: [pool www] child 849 said into stderr: "NOTICE: PHP message: PHP Fatal error:  Allowed memory
  size of 134217728 bytes exhausted ... in Unknown on line 0"
```

mod_php (Apache error_log):

LOG / PATTERN

```
[php:error] [pid 587] PHP Fatal error:  Allowed memory size of 134217728 bytes exhausted (tried to
  allocate 4096 bytes) in Unknown on line 0
```

Regex:

LOG / PATTERN

```
Allowed memory size of [0-9]+ bytes exhausted.* in Unknown on line 0
```

Yorum: `in Unknown on line 0`, hatada bir betik/satır konumu bulunmadığını gösterir; tek başına multipart ayrıştırıcısını veya saldırıyı kanıtlamaz. Lab'de startup-OOM sırasında gözlenmiştir. Aynı kaynağa güvenilir biçimde bağlanan, boyutu veya part sayısı artan tekrarlarla birlikte şüphe güçlenir.

A2 - İstek başlangıcında session ID uyarısı

LOG / PATTERN

```
PHP Warning:  PHP Request Startup: Session ID is too long or contains illegal characters ... in Unknown
  on line 0
```

`PHP Request Startup:` öneki başlangıç bağlamını belirtir. Bu session uyarısı lab'deki upload-progress yoluyla uyumludur, ancak önek tek başına belirli bir işleyiciyi veya RINIT sırasını kanıtlamaz. Yamalı sürümlerde ve meşru hatalı isteklerde de görülebilir. Beklenmeyen multipart alanlarıyla birlikte araştırın; yalnızca bu satırla HIGH/CRITICAL üretmeyin.

A3 - Session okuma hatası (A2 ile aynı zincir)

LOG / PATTERN

```
PHP Warning:  PHP Request Startup: Failed to read session data: files (path: /var/lib/php/sessions) in
  Unknown on line 0
```

Yamalı/yamasız ayrımı yapmaz. İzin, depolama veya session yapılandırma hataları da neden olabilir. A2, multipart bağlamı ve session depolama sağlığıyla birlikte değerlendirin.

A4 - İstek başlangıcında session yazma hatası

LOG / PATTERN

```
PHP Warning:  PHP Request Startup: Failed to write session data (files) ... in Unknown on line 0
```

`rfc1867_sid_invalid` regresyon senaryosunda yamasız sürüm, okuma başarısızken yazmayı denemiş; yamalı sürüm bunu yapmamıştır. **Bu test farkı, genel "Failed to write session data" mesajını zafiyetli sürüme özgü kılmaz.** Yamalı sistemde de bağımsız yazma/izin/depolama hataları görülebilir. Yazma başarılı olduğunda bu hata mesajı oluşmaz. Dolayısıyla A4'ün varlığı sürüm veya sömürü kanıtı, yokluğu güvenlik kanıtı değildir. Yama doğrulamasında paket sürümü, üretici duyurusu ve çalışan süreç bilgisi kullanılmalıdır.

A5 - Content-Length ve gövde limiti uyarıları

LOG / PATTERN

```
POST Content-Length of [0-9]+ bytes exceeds the limit of [0-9]+ bytes
Actual POST length does not match Content-Length, and exceeds [0-9]+ bytes
```

Bunlar iki farklı uyarı desenidir. İlk satır sıradan bir büyük POST'un limit tarafından reddedilmesinde de oluşur; tek başına sahte `Content-Length` veya başarılı bypass göstermez. İkinci satır gerçek/bildirilen uzunluk uyumsuzluğuna ve limit aşımına işaret eder. Proxy/WAF'ın aktarılan ve çözümlenmiş gövdeyi nasıl ölçtüğünü doğrulayın; HTTP framing veya proxy normalizasyonu şüpheli isteği PHP'ye ulaşmadan reddedebilir.

A6 - WAF / body inspection (log değil, kural)

Multipart gövdesinde, uygulama meşru olarak kullanmıyorsa şu alanları arayın:

- `name="PHP_SESSION_UPLOAD_PROGRESS"` form alanı
- `name="PHPSESSID"` gövdeden verilen SID
- Megabaytlarca yinelenen `Content-Disposition` / `Content-Type` başlığı; tek part'ta yüz binlerce başlık; `Content-Length: 0` + büyük gövde

Alan adları: `PHP_SESSION_UPLOAD_PROGRESS` ve `PHPSESSID` varsayılan adlardır; etkin `session.upload_progress.name` ve `session.name` ayarlarına uyarlayın. Alanın varlığı tek başına kötü niyet göstermez.

06 B grubu - Worker hataları ve araştırma sinyalleri

Bu kayıtlar servis hatası, kaynak tükenmesi veya olası bellek bozulmasını araştırmaya yardımcı olur. **Başarılı sömürünün göstergesi olarak tek başlarına kullanılamazlar.**

- **FPM** `exited with code 70`**:** İncelenen startup-OOM testlerinde worker bu kodla sonlandı. SIGSEGV değildir; başka startup hataları da üretebilir. OOM atfı için A1 ve worker bağlamıyla ilişkilendirin.
- **SIGSEGV / SIGABRT:** Bellek bozulmasıyla ilişkili olabilir; extension hataları, assertion'lar ve başka yazılım sorunları da üretebilir. Bir crash, heap grooming yapıldığını veya sömürünün başarıya ulaştığını kanıtlamaz.

Nerede	Ne aranır
FPM ana log	`child [0-9]+ exited with code 70` (OOM-startup) · `exited on signal 11 (SIGSEGV)` · `exited on signal 6 (SIGABRT)` · kısa aralıklı worker respawn kümeleri
Apache error log	`seg fault or similar nasty error detected in the child` · `child pid [0-9]+ exit signal Segmentation fault \(11\)`
nginx error log	`upstream prematurely closed connection` · `recv\(\) failed` + access log'da 502 burst'ları
Apache + FPM	`AH01071: Got error 'PHP message: ...'` (PHP mesajı Apache log'una taşınır)
Access log (korelasyon)	Aynı IP'den multipart POST → hemen ardından 500/502/503 → aynı IP'nin devam etmesi
Session dizini	Uygulamanın oluşturmayacağı `sess_` dosyaları; içeriğinde `upload_progress_` anahtarları
HTTP yanıtları	Anormal uzun / tuhaf karakterli `Set-Cookie: PHPSESSID=...` - memory disclosure vektörünün olası izi

Not: Başarılı **sessiz** sömürü (saf memory disclosure) hiç log üretmeyebilir. Bu yüzden B grubu + yanıt boyutu anomalileri birlikte izlenmeli. `Set-Cookie` ve response boyutu izleri bildirilmiş vektörlere dayanır; bizim labımızda memory disclosure üretilmediğinden bunları bildirilen (doğrulanmamış) sinyaller olarak ele alın.

07 Trafik ve access log ayak izi

HTTP status kodu tek başına yeterli değildir. Aşağıdaki gözlemler ORBITRON'un kontrollü testlerine aittir; genel saldırı imzası olarak yorumlanmamalıdır.

- **200 ve 404 dışlayıcı değildir:** Kontrollü preload/session testinde dosya yazan istek 200 dönebilmiş; var olmayan betiğe yöneltilen istekte 404 görülürken de startup kodu çalışmıştır. Buna karşılık büyük bir 404 POST isteği tek başına saldırı değildir. Normal erişim log'undan multipart part sayısı çıkarılamaz; WAF/body inspection gerekir.
- **FPM access log boşluğu:** İncelenen startup-OOM koşulunda error log ve `code 70` varken ilgili FPM access kaydı oluşmamıştır. Bu nedenle front-end nginx/Apache/WAF loglarını da toplayın. Eksik kayıt; log ayarı, filtreleme, rotasyon veya toplama kaybından da kaynaklanabilir. Tek başına startup ölümü, saldırı veya yamasızlık kanıtı değildir; eşleştirmede yalnızca yakın zaman damgasına güvenmeyin.
- **Kademeli istekler:** Aynı hedefe artan gövde/part sayısıyla gelen seri ve yükselen süre, kalibrasyon veya kaynak tüketimi açısından araştırılabilir. Lab'deki az sayıda ölçüm **O(n²)** karmaşıklığı kanıtlamaz. Eşik ve önem derecesini uygulamanın normal trafiğiyle belirleyin.
- **Hacim:** Düşük bellek limitinde birkaç yüz KB ile görülen OOM, 128M üretim sınırında belirli bir MB eşiğine doğrusal olarak çevrilemez. Alan yapısı, parser/SAPI, session ayarları ve derleme farkları sonucu değiştirir. Gövde boyutunu tek başına güvenlik eşiği yapmayın.
- **Session izleri:** Upload-progress, uygulama `session_start()` çağırmadan da session verisini güncelleyebilir. `sess_*` ve `upload_progress_` izleri meşru upload-progress kullanımında da oluşur; yalnızca ilgili özelliğin beklenmediği akışlarda ek bağlamla anlamlıdır.

Kontrollü dosya yazma ve bağımsız OOM testinin ayrımı

Test	Ön koşul	Gözlem	Kanıt sınırı
Dosya yazma: 8.4.26 / 8.4.27	Lab'de hazırlanmış `opcache.preload` sınıfı ve önceden yerleştirilmiş session nesnesi	Yamasızda startup sırasında `__wakeup` ve marker yazma; yamalı kontrolde marker oluşmadı	Hazırlanmış ortamdaki kod çalıştırma; tam uzaktan bellek bozulması zinciri değil
Ayrı multipart/OOM deneyi	Preload veya hazırlanmış session nesnesi gerektirmeyen istekler	Bellek tüketimi, worker çıkışı ve log görünürlüğü incelendi	Tek başına UAF veya RCE kanıtlamaz

Dosya yazma testinde `hello.txt` için **7.634 bayt, 22 ekleme, UID/GID 65534 ve 0644 izinleri** raporlandı. Dosya sahibi kimliği tek başına yazan süreci kanıtlamaz; olay telemetrisiyle PID, süreç başlangıç zamanı ve yürütülebilir dosya ilişkilendirilmelidir. Bu isim, boyut ve UID değerleri **lab artefaktıdır; üretim IOC'si değildir**.

08 C grubu - Sömürü sonrası avcılık

PHP worker kaynaklı beklenmedik süreç, dosya ve ağ etkinliğini araştırın. **PHP kendi dosya API'leriyle yeni süreç başlatmadan yazabilir; yalnızca `execve` izlemek dosya yazmayı kaçırır.**

- **auditd (süreç):** web worker kullanıcısının `euid`'siyle `execve` izleyin:

LOG / PATTERN

```
auditctl -a always,exit -F arch=b64 -S execve -F euid=48 -k php2shell_webexec
```

`/bin/sh`, `/bin/bash`, `curl`, `wget`, `base64`, `perl`, `python` çalıştırmaları (euid = apache genelde 48, www-data 33).

- **Dosya sistemi:** `/tmp`, `/dev/shm`, upload dizinleri ve webroot'ta yeni dosyalar - özellikle yeni `.php` (webshell) ve anlamsız isimli script dosyaları.
- **Kalıcılık:** web worker kullanıcısına ait crontab girişleri, SSH `authorized_keys`, systemd user unit'leri.
- **Ağ:** PHP worker'dan beklenmedik giden bağlantılar (reverse shell, C2, veri sızıntısı).

Toplama ve atıf: auditd kuralındaki `euid=48` örnektir; gerçek pool kullanıcısıyla değiştirin. Kural `execve` çağrılarını izler; dosya yazma ve `execveat` kapsamını sağlamaz. `php2shell_webexec` anahtarı repodaki Wazuh süreç kurallarıyla eşleşir. Audit kayıtlarını `/var/log/audit/audit.log` üzerinden alın; kalıcı kural için dağıtımınızın audit yapılandırmasını kullanın. Dosya yazmaları için EDR dosya olayları, hedef dizinlere uygun audit kuralları veya FIM ekleyin. FIM değişikliği gösterebilir; her ürün yazan PID'yi vermez. Windows'ta yapılandırılmış Sysmon Event ID 1/11 ve ürünün dosya/ağ telemetrisi karşılıklarını toplayın. Yeni bir `.txt` marker, yalnızca `.php` uzantısına odaklanan kurallarca kaçırılabilir.

Bu etkinlikleri değişiklik kayıtları ve uygulamanın normal davranışıyla karşılaştırın; meşru `exec()` kullanan uygulamalar (görsel/PDF araçları, yedekleme, deploy hook'ları) false positive üretir.

09 Korelasyon ve alarm mantığı

PHP hata satırında kaynak IP bulunmayabilir. İstemci IP'sini güvenilen proxy zincirinden alın; mümkünse request ID, host/container, pool, worker PID ve süreç başlangıç zamanıyla ilişkilendirin. Aynı PID zaman içinde yeniden kullanılabilir. Yalnızca yakın zaman damgası veya aynı pool, aynı isteğin kanıtı değildir.

Önerilen öncelik	Koşul	Yorum / aksiyon
HIGH	Güvenilir biçimde aynı kaynağa bağlanan A1 ≥3 / 10 dk ve olağandışı multipart örüntüsü	Tekrarlayan kaynak tüketimi veya olası deneme; hızlı triyaj
HIGH	Olağandışı multipart + ilişkili 5xx/worker yeniden başlatma + tekrar	Servis etkisi/olası deneme; başarı varsaymayın
MEDIUM	Beklenmeyen A2/A3 ve multipart bağlamı	Session/configuration/depolama nedenlerini de inceleyin
HIGH; doğrulanırsa CRITICAL	Worker'a atfedilmiş yetkisiz dosya/komut/ağ etkinliği	Değişiklik kayıtlarıyla doğrulayın; yetkisiz etkinlikte IR sürecini başlatın
MEDIUM	İlişkili A1 + FPM code 70	OOM kaynaklı startup sonlanmasıyla uyumlu
MEDIUM	Açıklanamayan SIGSEGV/SIGABRT	Crash incelemesi; ek bağlamla yükseltin
Araştırma sinyali	Error log'da startup hatası, FPM access kaydı yok	Önce log hattını doğrulayın; front-end kayıtlarıyla tamamlayın
MEDIUM	Aynı hedefe ≥4 artan boyutlu istek ve artan süre	Olası kalibrasyon; başlangıç eşiğidir, saldırı kanıtı değildir

Bu seviyeler başlangıç önerisidir; olay güveni ve operasyonel önem aynı şey değildir. Kritik servis kesintisi, sömürü doğrulanmadan da acil müdahale gerektirebilir.

Mükerrer olay: Aynı hata FPM, Apache ve journal üzerinden birden çok kez gelebilir. Tekilleştirmeden “3 deneme” gibi eşikler hesaplamayın.

FPM access log alanlarını doğru kaydedin

Pool yapılandırması için örnek; mevcut dizin, izin ve log rotasyonu politikanıza uyarlayın:

CONFIG

```
access.log = /var/log/php-fpm/www-access.log
access.format = "%R %t method=%m uri=%r status=%s content_length=%l duration_ms=%{milliseconds}d
  worker_pid=%p pool=%n script=%f"
```

`%p` child/worker PID, `%P` parent PID'dir. `%l` bildirilen POST Content-Length'tir; gerçek alınan gövde baytı değildir. `%{milliseconds}d` açıkça milisaniye üretir. `%t` zaman dilimini içerir; SIEM'de normalize edin. Gerçek alınan bayt için front-end telemetrisini kullanın ve alanın başlıkları da sayıp saymadığını kontrol edin. Request ID aktarımını proxy ve FPM arasında ayrıca yapılandırın. Bu format, startup sırasında hiç oluşmayan FPM access kaydını geri getirmez.

10 Hazır sorgular ve ORBITRON kuralları

Hızlı triyaj için grep'ler:

```shell
# Gerçek log yollarını yapılandırmadan doğrulayın.
# A1: PHP/FPM/Apache OOM
grep -E 'Allowed memory size of [0-9]+ bytes exhausted.*in Unknown on line 0' \
  /var/log/php-fpm/*log /var/log/php*-fpm.log \
  /var/log/httpd/*error* /var/log/apache2/*error*
```

```shell
# A2-A4: PHP Request Startup
grep -F 'PHP Request Startup:' \
  /var/log/php-fpm/*log /var/log/php*-fpm.log \
  /var/log/httpd/*error* /var/log/apache2/*error*
```

```shell
# FPM: code 70 / signal
grep -E 'exited with code 70|exited on signal (11|6)' \
  /var/log/php-fpm/*log /var/log/php*-fpm.log
```

```shell
# Apache + FPM
journalctl -u httpd -u apache2 --since today | \
  grep 'AH01071' | grep -E 'Allowed memory|Request Startup'
```

```shell
# nginx combined: POST + 5xx; multipart olduğunu göstermez.
awk '$6=="\"POST" && $9~/^50[0-9]$/' /var/log/nginx/access.log
```

ORBITRON php2shell-response-kit (public repo) karşılıkları:

Sinyal	Kural
A1 (OOM, in Unknown on line 0)	Sigma `php2shell_php_oom_during_request_parsing` · Wazuh 109627
OOM + ardından worker çökmesi (2 dk)	Wazuh 109628
OOM seri (5/5 dk)	Wazuh 109629
FPM worker exited with code 70 (OOM-startup)	Sigma `php2shell_php_fpm_worker_exit_code70` · Wazuh 109630
Code 70, bir OOM-during-parsing'i takip eden 2 dk içinde	Wazuh 109631
Worker çökmesi (SIGSEGV/SIGABRT)	Sigma `php2shell_php_worker_crash` · Wazuh 109625/109626
A6 başlık seli / body hatası / strict parsing hatası	ModSecurity 9109622 / 9109623 / 9109621
Upload-progress alanı	ModSecurity 9109618 / 9109619
C grubu (webserver → shell/webshell)	Sigma spawns_shell / file_written_to_webroot · Wazuh 109610-109613, 109615-109616, 109620-109621 · YARA

Test kapsamı: SIEM/EDR sorguları pySigma ile üretilmiştir; canlı platformlarda doğrulanmamıştır. Wazuh XML/regex kontrolleri yapılmış, canlı manager'a yükleme ve korelasyon kuralları test edilmemiştir. ModSecurity 9109623 tetiklenmesi test kapsamı dışındadır; script ve YARA sonuçlarının kapsamı repodaki `tests/TESTED.md` dosyasında yer alır. Windows scriptinin gerçek Windows/IIS/PowerShell 5.1 doğrulaması da tamamlanmamıştır. Kısa bir baseline ilk triyaja yardımcı olur; 1-2 saat tüm meşru iş yüklerini temsil etmeyebilir.

Korelasyon sınırı: Repodaki Wazuh 109628/109631 kuralları, önceki OOM ile sonraki olayı aynı worker PID'sine bağlayan alan koşulu içermez. Bunları “aynı worker'da OOM doğrulandı” şeklinde raporlamayın. Host/agent kapsamını canlı manager'da sınayın; kesin süreç atfı için decoder ve PID/pool korelasyonu gerekir. Bu rehberin öncelik önerileri mevcut kural seviyelerini otomatik değiştirmez.

Yukarıdaki grep örnekleri döndürülmüş/sıkıştırılmış logları kapsamaz. Sistemde olmayan yollar hata verir; gerçek kaynak listesini uyarlayın. `awk` yalnızca standart combined alan sırasını varsayar ve multipart gövdesini incelemez.

11 Geçici önlemler, yama doğrulama ve kaynaklar

Öncelik: İki CVE'yi gideren upstream sürümü veya doğrulanmış vendor backport'unu uygulayın. İlgili servisleri ve uzun ömürlü worker'ları yeniden başlatın; yalnızca diskteki CLI sürümünü değil, web isteğini karşılayan süreçleri doğrulayın. Container kullanıyorsanız yamalı image'ı yeniden dağıtın ve eski pod/container'ların sonlandığını kontrol edin.

Aşağıdakiler **kısmi geçici önlemlerdir; yamanın yerine geçmez**. Web SAPI'nin etkin ayarlarına uygulanmalıdır. FPM pool örneği:

CONFIG

```
php_admin_value[memory_limit] = -1
php_admin_value[post_max_size] = 8M
php_admin_flag[session.upload_progress.enabled] = off
; Sadece PHP 8.5:
php_admin_flag[fatal_error_backtraces] = off
```

Upload gerektiren uygulamalarda PHP duyurusundaki 32M `post_max_size` önerisini iş yüküne göre değerlendirin; `upload_max_filesize` daha düşük olmalıdır. Sonradan `ini_set()` ile değişiklik startup yoluna yetişmez. Başka pool/dizin ayarlarının sonlu bellek limitini geri getirmediğini doğrulayın. Etkinleştirmeden önce yapılandırmayı test edin, bütün worker'ları yenileyin ve ayarların web SAPI'de geçerli olduğunu kontrol edin.

Önlem	İzlemeye etkisi ve sınırı
`session.upload_progress.enabled=Off`	İlgili erken session yolunu kapatır; aynı genel mesajın diğer nedenleri ve diğer CVE yolları kalır
`memory_limit=-1` + kaynak sınırları	PHP limitinden doğan A1 azalır; tahsis hatası riski ve DoS ortadan kalkmaz. Kernel/cgroup OOM ve yeniden başlamaları izleyin
Düşük `post_max_size` + proxy/body/part/header limitleri	Kapsanan istekleri reddeder; her saldırı yolunu engellemez. Kullanılabilir Content-Length olmadan gelen veriyi de sınırlayın
Yalnızca gerekli PHP betiklerine yönlendirme	PHP'ye ulaşmadan reddedilen gereksiz yolları azaltabilir; front-controller/rewrite davranışını doğrulayın
`fatal_error_backtraces=Off` (8.5)	Doğrulanmış bir stale-state üreticisini kaldırır; diğer durum yollarını kapatmaz
Geçersiz/deprecated erken INI ayarlarını düzeltme	Tanı üreten bir erken tüketici yolunu azaltır; yalnızca uyarıları gizlemek koruma sağlamaz

Görünürlük: PHP hata kaydı, FPM process/access kaydı, web/WAF kaydı ve endpoint dosya/süreç/ağ olaylarını birlikte toplayın. `display_errors=Off` loglamayı kapatmaz; `log_errors` ve ilgili hata seviyeleri etkin

olmalıdır. `catch_workers_output` yalnızca worker stdout/stderr yönlendirmesine yardımcı olur, dosyaya yazılan PHP hata logunun yerine geçmez. Mevcut kanıtları koruyun; yetkisiz etkinlik doğrulanırsa IR sürecinde izolasyon ve kapsam araştırması yapın.

Kaynaklar:

- PHP güvenlik sürümleri 8.5.12 / 8.4.27 / 8.3.36 / 8.2.35 (21 Ekim 2026): php.net/archive/2026
- Düzeltme commit'i "Avoid state corruption via bailout" (GHSA-w8hw-w9fx-j79w, GHSA-6wmf-4rv5-p3rp): github.com/php/php-src
- ORBITRON php2shell-response-kit (envanter script'leri + kurallar): github.com/orbitron-ai/php2shell-response-kit
- Resmî CVE duyuruları: OOM corruption, stale global state
- FPM log alanları: PHP FPM configuration
- Kural test kapsamı: TESTED.md

Lab doğrulaması, ORBITRON tarafından 21 Ekim 2026'da izole ortamda, PHP'nin resmi kaynağından derlenen yamasız/yamalı sürümler ve resmi regresyon testleriyle yapılmıştır. Exploit zinciri üretilmemiş, üretim sistemlerinde tetiklenmemiştir.

12 Belge kontrolü

Belge sahibi: **ORBITRON Ekibi** · Belge kodu: **ORB-PHP2SHELL-TR**

Sürüm	Tarih	Değişiklik
1.0	21.10.2026	İlk kurumsal tasarım ve belge kontrolü
1.1	21.10.2026	Kontrollü dosya yazma, OOM ayrımı ve telemetri ekleri
1.2	21.10.2026	GitHub final metinleriyle eşitleme; kanıt sınırları, FPM kapsamı, kural eşleştirmeleri ve toplama örnekleri düzeltildi